

1 Commitment Schemes

A *commitment scheme* is a cryptographic primitive that allows a party to commit to a value while keeping the value hidden. Later, the party can reveal the value and provide the information required to verify that it is the same value to which the party originally committed.

A commitment scheme therefore provides two fundamental security properties:

1. **Hiding:** Given the commitment, the receiver should not be able to learn the committed value.
2. **Binding:** After producing a commitment, the sender should not be able to open the same commitment to two different values.

The commitment process consists of two phases:

1. **Commit phase:** The sender chooses a value m and produces a commitment c to m , typically using some random value r . The sender sends c to the receiver.
2. **Opening phase:** The sender reveals m and the corresponding opening information, such as r . The receiver verifies that the opening is consistent with the original commitment c .

Thus, a commitment can be viewed as a cryptographic equivalent of putting a value inside a locked box: the receiver cannot see the value before the box is opened, and the sender cannot change the value after the box has been locked.

1.1 Example: Committing to a Future Grade

Consider a teacher who wants to make a decision about a student's final grade but does not want to reveal the decision immediately. For example, the teacher wants to commit to whether the student's final grade will be at least B .

Let $b \in \{0, 1\}$ denote the decision, where $b = 1$ means that the final grade will be at least B , and $b = 0$ means otherwise.

Before revealing the decision, the teacher generates a commitment to b and sends the commitment to the student. At this stage, the student should not be able to determine whether the committed value is 0 or 1.

After the final grade is determined, the teacher reveals b together with the information required to verify the commitment. The student can then check that the revealed value is consistent with the commitment sent earlier.

The commitment provides two guarantees. First, the student's inability to determine b before the opening provides the *hiding* property. Second, once the commitment has been made, the teacher cannot change the decision from $b = 0$ to $b = 1$, or vice versa, while still producing a valid opening. This provides the *binding* property.

Thus, the commitment allows the teacher to make a decision now and reveal it later, while ensuring that the decision remains both hidden and fixed.

2 Hash-Based Commitment

A simple commitment scheme can be constructed using a cryptographic hash function H .

The sender chooses a random value $r \leftarrow \{0, 1\}^\lambda$ and computes

$$c = H(m \| r).$$

Only c is sent to the receiver.

The random value r acts as a hiding value. Even if the message m comes from a small set, the receiver should not be able to determine which message was committed by observing the commitment alone.

To open the commitment, the sender reveals (m, r) . The receiver computes $H(m \| r)$ and checks whether it equals the original commitment c .

If the values match, the receiver accepts the opening.

2.1 Why is the Commitment Binding?

Suppose the sender initially committed using (m, r) but later attempts to open the same commitment using a different message $m' \neq m$ and some value r' .

For the alternative opening to be accepted, it must satisfy

$$H(m\|r) = H(m'\|r').$$

Finding such an alternative opening should be computationally infeasible for a suitable cryptographic hash function.

Thus, the binding property follows from the collision resistance of the hash function.

2.2 Why is the Commitment Hiding?

The receiver sees only $c = H(m\|r)$ and does not know the random value r . For a suitable cryptographic hash function and sufficiently large randomness, the receiver should not be able to determine m from c .

The random value is particularly important when the message space is small. Without randomness, an attacker could simply compute the hash of every possible message and compare it with the commitment.

3 Committing to Different Values

The commitment need not be restricted to a single bit. For example, suppose the student's final mark is $m \in [0, 100]$. The teacher can directly commit to the mark by computing

$$c = H(m\|r).$$

The teacher later reveals (m, r) , and the student verifies the commitment by recomputing the hash.

Thus, the commitment mechanism can be used for a bit, an integer, a message, or other types of data.

4 Categories of Commitment Schemes

Commitment schemes can be classified in two different and largely independent ways.

The first classification is based on *how the commitment is constructed*. Examples include:

- Hash-based commitments.
- Discrete-logarithm-based commitments.
- RSA-based commitments.
- Lattice-based commitments.
- Code-based commitments.

The second classification is based on *what is being committed*. Examples include:

- Bit commitments.
- Message or string commitments.
- Integer or numerical commitments.
- Vector commitments.
- Polynomial commitments.
- Set commitments.

The first classification concerns the underlying cryptographic construction and its security assumption. The second concerns the mathematical structure of the data being committed.

These two classifications are important because the choice of commitment scheme depends on both the security requirements and the type of computation that needs to be performed on the committed data. Different constructions provide different efficiency, security, and functionality properties.

For example, a simple hash-based commitment is sufficient when the main requirement is to commit to a message and later open it. However, more specialized commitments may be required when we need to prove properties of the committed data, perform efficient verification, or work with structured objects such as vectors or polynomials.

The underlying security assumption is also important. A commitment based on the discrete logarithm problem is efficient in a classical setting, but it is not secure in the presence of quantum computers. This motivates the study of post-quantum commitment schemes based on assumptions believed to remain secure against quantum adversaries.

Thus, in the following sections, we consider commitment schemes from two perspectives: *what is being committed* and *how the commitment is constructed*.

5 What Can Be Committed?

A commitment scheme can be used to commit to objects with different mathematical structures. Depending on the application, the committed object may be a single bit, a message, an integer, a vector, a polynomial, or a set.

For example, a vector can be represented as

$$\mathbf{x} = (x_1, \dots, x_n),$$

and a polynomial of degree at most d can be represented as

$$f(X) = \sum_{i=0}^d a_i X^i.$$

The choice of commitment scheme depends not only on the object being committed, but also on what we want to do with the commitment later. In some applications, it is sufficient to commit to a value and later open it. In other applications, we may want to prove properties of the committed object without revealing the entire object, or efficiently verify individual components of a larger object.

It is therefore important to distinguish between the *committed object* and the *commitment mechanism*. The committed object specifies what is being hidden, while the commitment mechanism specifies how it is cryptographically committed.

6 Committing to Different Types of Data

In practice, the data to be committed may not initially have the form required by a particular commitment scheme. It may be a message, an integer, a structured object, or some other data type. Such data can be encoded into an appropriate representation before applying the commitment mechanism.

For example, an object x can be represented as a binary string using an encoding function

$$\text{Encode}(x) \in \{0, 1\}^*.$$

A hash-based commitment can then be constructed as

$$C = H(\text{Encode}(x) \| r),$$

where r is a randomly chosen nonce.

Conceptually, the process can be viewed as

$$\text{Data} \longrightarrow \text{Encoding} \longrightarrow \text{Commitment}.$$

However, committing to data is often only the first step. In many cryptographic applications, we also want to perform some operation or prove a property of the committed data without revealing it. For example, we may want to prove that a committed integer lies within a particular range, that a committed vector satisfies a relation, or that a committed polynomial has a particular property.

Therefore, the choice of a commitment scheme is determined not only by how the data is represented, but also by the operations and proofs that need to be performed on the committed data.

7 Commitment versus Zero-Knowledge Proof

Commitment schemes and zero-knowledge proofs both allow a party to protect private information while establishing certain guarantees, but they serve different purposes.

In a commitment scheme, a party first commits to a value while keeping it hidden. At a later stage, the party opens the commitment by revealing the value and the corresponding opening information. The commitment guarantees that the value was fixed at the time of commitment and was not changed before the opening. In contrast, a zero-knowledge proof allows a prover to establish that a hidden value satisfies a particular property without revealing the value itself.

For example, suppose a teacher commits to a student's mark m . If the teacher later opens the commitment, the student learns the exact value of m .

Now suppose the teacher wants to prove only that the student's mark satisfies

$$m \geq 50,$$

without revealing the actual mark. Simply opening the commitment is not suitable because it reveals m completely.

This motivates a more general question:

Can a party prove that a committed value satisfies a particular property without revealing the value itself?

Zero-knowledge proofs provide a way to achieve this. The prover can demonstrate that the hidden value satisfies the required property while keeping the value itself secret.

8 From Commitments to Zero-Knowledge Proofs

Suppose a value m is committed as

$$C = \text{Com}(m; r).$$

Instead of opening the commitment and revealing m , we may want to prove that m satisfies a particular property. For example, the prover may want to prove that

$$50 \leq m \leq 100$$

without revealing the exact value of m .

More generally, for a public statement x and a private witness w , a zero-knowledge proof allows the prover to establish that

$$R(x, w) = 1$$

without revealing w .

Thus, commitments hide and fix a value, while zero-knowledge proofs allow properties of hidden values to be proved without revealing the values themselves. Commitments are therefore important building blocks for many zero-knowledge protocols.